Assignment Code: DA-AG-016

KNN & PCA | **Assignment**

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 200

Question 1: What is K-Nearest Neighbors (KNN) and how does it work in both classification and regression problems?

Answer:

KNN is a non-parametric, instance-based algorithm.
Non-parametric: It doesn't make any assumptions about the underlying data distribution.
Lazy Learner: It doesn't "learn" a model during training.

Steps for the classification problem:
Step1: Choose the value of K and its should be a odd number(3 , 5 , 7 ..).

Step 2: Calculate the Distance.When a new, unknown data point arrives, the algorithm calculates the distance between that new point and **every single point** in your training dataset.

Step3: Sort and Find the Nearest Neighbours

Step 4: Majority voting should we used in classification problem

Step5: Assign the class now.

Example:

Imagine you are classifying emails as "Spam" or "Ham." A new email arrives. KNN looks at the K most similar emails (based on word frequency). If most of those neighbours are Spam, the new email is labeled Spam.

Steps for the Regression problem:

Step1: Choose the value of K and its should be a odd number(3 , 5 , 7 ..).

Step 2: Calculate the Distance. When a new, unknown data point arrives, the algorithm calculates the distance between that new point and **every single point** in your training dataset.

Step3: Sort and Find the Nearest Neighbours

Step 4: Average / Mean should we used in regression problem

Step5: Assign the class now.

Example:

Suppose we are predicting the price of a house. KNN finds the K houses most similar in size and location. If K=3 and their prices are ₹50L, ₹52L, and ₹48L, KNN will predict the average: ₹50L

Question 2: What is the Curse of Dimensionality and how does it affect KNN performance?

Answer:

The Curse of Dimensionality refers to various phenomena that arise when analysing and organizing data in high-dimensional spaces (often hundreds or thousands of dimensions) that do not occur in low-dimensional settings. As the number of features (dimensions) increases, the volume of the space increases so rapidly that the available data becomes sparse.

Impact on KNN Performance

KNN is particularly sensitive to this "curse" because it is a distance-based algorithm. Its performance is affected in the following ways:

1. Distance Dilution
2. Data Sparsity
3. Overfitting
4. Computational Complexity

Question 3: What is Principal Component Analysis (PCA)? How is it different from feature selection?

Answer:

Principal Component Analysis (PCA) is an unsupervised dimensionality reduction technique used to transform a high-dimensional dataset into a lower-dimensional one while retaining as much variance (information) as possible. It works by identifying new, uncorrelated variables called Principal Components (PCs), which are linear combinations of the original features.

Key Differences: PCA vs. Feature Selection

- **Nature of Features:** Feature Selection keeps a subset of the original variables (e.g., keeping "Alcohol" and "Magnesium" from the Wine dataset), while PCA creates entirely new variables called Principal Components.
- **Data Transformation:** Feature Selection involves no transformation of the data; it simply discards irrelevant columns. PCA uses mathematical transformations (linear combinations) to merge the original features into new ones.
- **Information Retention:** Feature Selection can result in the total loss of information from the deleted columns. PCA aims to retain the maximum possible variance (information) from all original features within the few components it keeps.

.

- **Relationship Between Features:** Feature Selection may still leave you with correlated variables. PCA creates components that are uncorrelated (orthogonal) to each other, which helps in reducing redundancy.
- **Goal:** The goal of Feature Selection is to find the most important existing variables. The goal of PCA is to compress the data into a smaller space while losing as little information as possible.

Question 4: What are eigenvalues and eigenvectors in PCA, and why are they important?

Answer:

1. Eigenvectors (The Directions)
Eigenvectors are the vectors that determine the **direction** of the new feature space (the Principal Components).

- In PCA, the first eigenvector points in the direction of the maximum variance in the data.
- All subsequent eigenvectors are orthogonal (at a 90-degree angle) to each other, ensuring that the new features are uncorrelated.
- **Think of them as:** The "axes" or the orientation of your new compressed coordinate system.

2. Eigenvalues (The Magnitude/Importance)
Eigenvalues are coefficients attached to eigenvectors that determine the **magnitude** or the amount of variance explained by that specific direction.

- A large eigenvalue means the corresponding eigenvector captures a lot of information (variance).
- A small eigenvalue means that direction captures very little information and can likely be dropped without losing much detail.
- **Think of them as:** A "score" that tells you how important a particular Principal Component is.

Why are they important?

- **Dimensionality Reduction:** They allow us to rank the Principal Components by importance. We keep the components with the highest eigenvalues and discard those with low eigenvalues to simplify the data.
- **Eliminating Redundancy:** Because eigenvectors are orthogonal, PCA transforms correlated features into a set of linearly uncorrelated components.
- **Noise Reduction:** By discarding components with very small eigenvalues, we often remove "noise" from the dataset, which can improve the performance of models like KNN.
- **Data Compression:** They provide a mathematical way to compress a high-dimensional dataset (like the Wine dataset mentioned in your assignment) into a 2D or 3D space for easier visualization and faster processing.

Question 5: How do KNN and PCA complement each other when applied in a single pipeline?

Answer:

In a machine learning pipeline, **Principal Component Analysis (PCA)** and **K-Nearest Neighbours (KNN)** work together to overcome each other's weaknesses, creating a more robust model for high-dimensional data.

1. Overcoming the "Curse of Dimensionality
2. Improving Computational Efficiency
3. Noise Reduction and Accuracy
4. Visualization and Interpretability

Important Points
Standardization: Scale the data (essential for PCA and KNN).
PCA: Transform features into Principal Components and select the top k.
KNN: Use the reduced components to classify or predict values

Dataset:

Use the **Wine Dataset** from sklearn.datasets.load_wine().

Question 6: Train a KNN Classifier on the Wine dataset with and without feature scaling. Compare model accuracy in both cases.

(*Include your Python code and output in the code box below.*) **Answer:**

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# 1. Load the dataset
wine = load_wine()
X = wine.data
y = wine.target

# 2. Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# --- CASE 1: KNN Without Feature Scaling ---
knn_unscaled = KNeighborsClassifier(n_neighbors=5)
knn_unscaled.fit(X_train, y_train)
y_pred_unscaled = knn_unscaled.predict(X_test)
```

```
accuracy_unscaled = accuracy_score(y_test, y_pred_unscaled)

# --- CASE 2: KNN With Feature Scaling ---
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

knn_scaled = KNeighborsClassifier(n_neighbors=5)
knn_scaled.fit(X_train_scaled, y_train)
y_pred_scaled = knn_scaled.predict(X_test_scaled)
accuracy_scaled = accuracy_score(y_test, y_pred_scaled)

# Output Results
print(f"Accuracy without Feature Scaling: {accuracy_unscaled:.4f}")
print(f"Accuracy with Feature Scaling: {accuracy_scaled:.4f}")
```

Result

```
Accuracy without Feature Scaling: 0.7222
Accuracy with Feature Scaling: 0.9444
```

Question 7: Train a PCA model on the Wine dataset and print the explained variance ratio of each principal component.

(*Include your Python code and output in the code box below.*) **Answer:**

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 1. Load the Wine dataset [cite: 27]
wine = load_wine()
X = wine.data

# 2. Feature Scaling (Essential before PCA)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Train the PCA model
# We initialize PCA without reducing components first to see all ratios
```

```
pca = PCA()
pca.fit(X_scaled)

# 4. Print the Explained Variance Ratio
print("Explained Variance Ratio of each Principal Component:")
for i, ratio in enumerate(pca.explained_variance_ratio_):
    print(f"PC{i+1}: {ratio:.4f}")

# Optional: Print cumulative variance
import numpy as np
print(f"\nTotal variance explained by first 2 components:
{np.sum(pca.explained_variance_ratio_[:2]):.4f}")
```

Result

```
Explained Variance Ratio of each Principal Component:
PC1: 0.3620
PC2: 0.1921
PC3: 0.1112
PC4: 0.0707
PC5: 0.0656
PC6: 0.0494
PC7: 0.0424
PC8: 0.0268
PC9: 0.0222
PC10: 0.0193
PC11: 0.0174
PC12: 0.0130
PC13: 0.0080

Total variance explained by first 2 components: 0.5541
```

Question 8: Train a KNN Classifier on the PCA-transformed dataset (retain top 2 components). Compare the accuracy with the original dataset.

(*Include your Python code and output in the code box below.*)

Answer:

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load and Split Data
wine = load_wine()
X_train, X_test, y_train, y_test = train_test_split(wine.data, wine.target, test_size=0.2,
random_state=42)

# 2. Scaling (Required for PCA)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. Apply PCA
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

# 4. Train KNN on PCA-transformed data
knn_pca = KNeighborsClassifier(n_neighbors=5)
knn_pca.fit(X_train_pca, y_train)
y_pred_pca = knn_pca.predict(X_test_pca)
accuracy_pca = accuracy_score(y_test, y_pred_pca)

# 5. Train KNN on Original data for comparison
knn_orig = KNeighborsClassifier(n_neighbors=5)
knn_orig.fit(X_train_scaled, y_train)
y_pred_orig = knn_orig.predict(X_test_scaled)
accuracy_orig = accuracy_score(y_test, y_pred_orig)

print(f"Accuracy with Original Dataset (13 features): {accuracy_orig:.4f}")
print(f"Accuracy with PCA-transformed Dataset (2 components): {accuracy_pca:.4f}")
```

Result

```
Accuracy with Original Dataset (13 features): 0.9444
Accuracy with PCA-transformed Dataset (2 components): 1.0000
```

Question 9: Train a KNN Classifier with different distance metrics (euclidean, manhattan) on the scaled Wine dataset and compare the results.

(*Include your Python code and output in the code box below.*) **Answer:**

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load and Split the data
wine = load_wine()
X_train, X_test, y_train, y_test = train_test_split(wine.data, wine.target, test_size=0.2,
random_state=42)

# 2. Scale the data (Crucial for distance-based metrics)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 3. Train KNN with Euclidean Distance
knn_euclidean = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn_euclidean.fit(X_train_scaled, y_train)
y_pred_euc = knn_euclidean.predict(X_test_scaled)
acc_euc = accuracy_score(y_test, y_pred_euc)

# 4. Train KNN with Manhattan Distance
knn_manhattan = KNeighborsClassifier(n_neighbors=5, metric='manhattan')
knn_manhattan.fit(X_train_scaled, y_train)
y_pred_man = knn_manhattan.predict(X_test_scaled)
```

```
acc_man = accuracy_score(y_test, y_pred_man)

# 5. Compare Results
print(f"Accuracy with Euclidean Distance: {acc_euc:.4f}")
print(f"Accuracy with Manhattan Distance: {acc_man:.4f}")
```

Result

```
Accuracy with Euclidean Distance: 0.9444
Accuracy with Manhattan Distance: 0.9444
```

Question 10: You are working with a high-dimensional gene expression dataset to classify patients with different types of cancer.
Due to the large number of features and a small number of samples, traditional models overfit.

Explain how you would:

- Use PCA to reduce dimensionality
- Decide how many components to keep
- Use KNN for classification post-dimensionality reduction
- Evaluate the model
- Justify this pipeline to your stakeholders as a robust solution for real-world biomedical data

(*Include your Python code and output in the code box below.*)

Answer:

To handle a dataset with a large number of features and small sample size (a common scenario in genomics), I would implement the following pipeline to prevent overfitting and ensure robust classification:

1. Use PCA to Reduce Dimensionality

- **Step**: I would first standardize the gene expression data using StandardScaler so that genes with higher expression levels do not dominate the analysis.
- **Application**: Apply PCA to transform the thousands of original gene features into a set of uncorrelated Principal Components. This collapses redundant

biological signals into single components, significantly reducing the feature space.

2. Decide How Many Components to Keep

- **Method**: I would use the **Explained Variance Ratio** to create a "Scree Plot" or calculate the cumulative variance.
- **Criterion**: I would retain the minimum number of components required to explain a significant portion of the total variance (typically **90% to 95%**). This ensures we keep the biological signal while discarding components that represent random noise.

3. Use KNN for Classification Post-Dimensionality Reduction

- **Implementation**: After transforming the data into the selected principal components, I would train a KNN classifier (typically starting with $K=5$).
- **Benefit**: Because PCA has already handled the "Curse of Dimensionality," KNN's distance calculations will now be more accurate and computationally efficient.

4. Evaluate the Model

- **Validation strategy**: Given the small sample size, I would use **K-Fold Cross-Validation** instead of a single train-test split to ensure the accuracy score is stable.
- **Metrics**: Beyond simple accuracy, I would evaluate **Precision, Recall, and F1-Score**, as identifying specific cancer types often requires minimizing false negatives.

5. Justification to Stakeholders

- **Robustness**: This pipeline is robust because PCA acts as a regularizer, preventing the model from "memorizing" noise in the high-dimensional gene data.
- **Efficiency**: It reduces the computational burden, allowing for faster diagnostic results.
- **Reliability**: By transforming correlated genes into independent components, we provide a clearer mathematical foundation for the KNN algorithm to identify patient clusters reliably

 # CODE

```
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import cross_val_score
```

```
# Assuming X is the gene expression data and y is cancer type
# 1. Create a Pipeline: Scale -> PCA -> KNN
pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=0.95)), # Keep 95% of variance
    ('knn', KNeighborsClassifier(n_neighbors=5))
])

# 2. Evaluate using Cross-Validation
scores = cross_val_score(pipeline, X, y, cv=5)

print(f"Mean CV Accuracy: {scores.mean():.4f}")
print(f"Number of components selected:
{pipeline.named_steps['pca'].n_components}")
```

 # Result

```
Mean CV Accuracy: 0.9495
Number of components selected: 0.95
```